

[INICIAL] TCArg 2026 Contest #8

A. Shifting Stacks

1 second, 256 megabytes

You have n stacks of blocks. The i -th stack contains h_i blocks and it's height is the number of blocks in it. In one move you can take a block from the i -th stack (if there is at least one block) and put it to the $i + 1$ -th stack. Can you make the sequence of heights strictly increasing?

Note that the number of stacks always remains n : stacks don't disappear when they have 0 blocks.

Input

First line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \leq n \leq 100$). The second line of each test case contains n integers h_i ($0 \leq h_i \leq 10^9$) — starting heights of the stacks.

It's guaranteed that the sum of all n does not exceed 10^4 .

Output

For each test case output YES if you can make the sequence of heights strictly increasing and NO otherwise.

You may print each letter in any case (for example, YES, Yes, yes, yEs will all be recognized as positive answer).

input
6 2 1 2 2 1 0 3 4 4 4 2 0 0 3 0 1 0 4 1000000000 1000000000 1000000000 1000000000
output
YES YES YES NO NO YES

In the first test case there is no need to make any moves, the sequence of heights is already increasing.

In the second test case we need to move one block from the first stack to the second. Then the heights become 0 1.

In the third test case we could move one block from the first stack to the second and then from the second to the third, which would make the heights 3 4 5.

In the fourth test case we can't make a move, but the sequence is not increasing, so the answer is NO.

In the fifth test case we can only make one move (from the second to the third stack), which would make the heights 0 0 1. Both 0 1 0 and 0 0 1 are not increasing sequences, so the answer is NO.

B. Turn the Rectangles

2 seconds, 256 megabytes

There are n rectangles in a row. You can either turn each rectangle by 90 degrees or leave it as it is. If you turn a rectangle, its width will be height, and its height will be width. Notice that you can turn any number of rectangles, you also can turn all or none of them. **You can not change the order of the rectangles.**

Find out if there is a way to make the rectangles go in order of non-ascending height. In other words, after all the turns, a height of every rectangle has to be not greater than the height of the previous rectangle (if it is such).

Input

The first line contains a single integer n ($1 \leq n \leq 10^5$) — the number of rectangles.

Each of the next n lines contains two integers w_i and h_i ($1 \leq w_i, h_i \leq 10^9$) — the width and the height of the i -th rectangle.

Output

Print "YES" (without quotes) if there is a way to make the rectangles go in order of non-ascending height, otherwise print "NO".

You can print each letter in any case (upper or lower).

input
3 3 4 4 6 3 5
output
YES

input
2 3 4 5 5
output
NO

In the first test, you can rotate the second and the third rectangles so that the heights will be [4, 4, 3].

In the second test, there is no way the second rectangle will be not higher than the first one.

C. Nice Garland

1 second, 256 megabytes

You have a garland consisting of n lamps. Each lamp is colored red, green or blue. The color of the i -th lamp is s_i ('R', 'G' and 'B' — colors of lamps in the garland).

You have to recolor some lamps in this garland (recoloring a lamp means changing its initial color to another) in such a way that the obtained garland is **nice**.

A garland is called **nice** if any two lamps of the same color have distance divisible by three between them. I.e. if the obtained garland is t , then for each i, j such that $t_i = t_j$ should be satisfied $|i - j| \bmod 3 = 0$. The value $|x|$ means absolute value of x , the operation $x \bmod y$ means remainder of x when divided by y .

For example, the following garlands are **nice**: "RGBRGBRG", "GB", "R", "GRBGRBG", "BRGBRGB". The following garlands are not **nice**: "RR", "RGBG".

Among all ways to recolor the initial garland to make it **nice** you have to choose one with the **minimum** number of recolored lamps. If there are multiple optimal solutions, print **any** of them.

Input
The first line of the input contains one integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of lamps.
The second line of the input contains the string s consisting of n characters 'R', 'G' and 'B' — colors of lamps in the garland.

Output
In the first line of the output print one integer r — the **minimum** number of recolors needed to obtain a **nice** garland from the given one.
In the second line of the output print one string t of length n — a **nice** garland obtained from the initial one with **minimum** number of recolors. If there are multiple optimal solutions, print **any** of them.

input
3 BRB
output
1 GRB

input
7 RGBGRBB
output
3 RGBRGBR

D. Resort

2 seconds, 256 megabytes

Valera's finally decided to go on holiday! He packed up and headed for a ski resort.

Valera's fancied a ski trip but he soon realized that he could get lost in this new place. Somebody gave him a useful hint: the resort has n objects (we will consider the objects indexed in some way by integers from 1 to n), each object is either a hotel or a mountain.

Valera has also found out that the ski resort had multiple ski tracks. Specifically, for each object v , the resort has at most one object u , such that there is a ski track built from object u to object v . We also know that no hotel has got a ski track leading from the hotel to some object.

Valera is afraid of getting lost on the resort. So he wants you to come up with a path he would walk along. The path must consist of objects $v_1, v_2, \dots, v_k$ ($k \geq 1$) and meet the following conditions:

- 1. Objects with numbers $v_1, v_2, \dots, v_{k-1}$ are mountains and the object with number v_k is the hotel.
- 2. For any integer i ($1 \leq i < k$), there is **exactly one** ski track leading from object v_i . This track goes to object v_{i+1} .
- 3. The path contains as many objects as possible (k is maximal).

Help Valera. Find such path that meets all the criteria of our hero!

Input

The first line contains integer n ($1 \leq n \leq 10^5$) — the number of objects.

The second line contains n space-separated integers $type_1, type_2, \dots, type_n$ — the types of the objects. If $type_i$ equals zero, then the i -th object is the mountain. If $type_i$ equals one, then the i -th object is the hotel. It is guaranteed that at least one object is a hotel.

The third line of the input contains n space-separated integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq n$) — the description of the ski tracks. If number a_i equals zero, then there is no such object v , that has a ski track built from v to i . If number a_i doesn't equal zero, that means that there is a track built from object a_i to object i .

Output

In the first line print k — the maximum possible path length for Valera. In the second line print k integers $v_1, v_2, \dots, v_k$ — the path. If there are multiple solutions, you can print any of them.

input
5 0 0 0 0 1 0 1 2 3 4
output
5 1 2 3 4 5

input
5 0 0 1 0 1 0 1 2 2 4
output
2 4 5

input
4 1 0 0 0 2 3 4 2
output
1 1

E. Worms Evolution

2 seconds, 256 megabytes

Professor Vasechkin is studying evolution of worms. Recently he put forward hypotheses that all worms evolve by division. There are n forms of worms. Worms of these forms have lengths $a_1, a_2, \dots, a_n$. To prove his theory, professor needs to find 3 different forms that the length of the first form is equal to sum of lengths of the other two forms. Help him to do this.

Input

The first line contains integer n ($3 \leq n \leq 100$) — amount of worm's forms. The second line contains n space-separated integers a_i ($1 \leq a_i \leq 1000$) — lengths of worms of each form.

Output

Output 3 distinct integers $i\ j\ k$ ($1 \leq i, j, k \leq n$) — such indexes of worm's forms that $a_i = a_j + a_k$. If there is no such triple, output -1. If there are several solutions, output any of them. It possible that $a_j = a_k$.

input
5 1 2 3 5 7
output
3 2 1

input
5 1 8 1 5 1
output
-1

F. Inversions problem

2 seconds, 256 megabytes

You are given a permutation of n numbers $p_1, p_2, \dots, p_n$. We perform k operations of the following type: choose uniformly at random two indices l and r ($l \leq r$) and reverse the order of the elements $p_l, p_{l+1}, \dots, p_r$. Your task is to find the expected value of the number of inversions in the resulting permutation.

Input
The first line of input contains two integers n and k ($1 \leq n \leq 100, 1 \leq k \leq 10^9$). The next line contains n integers $p_1, p_2, \dots, p_n$ — the given permutation. All p_i are different and in range from 1 to n .

The problem consists of three subproblems. The subproblems have different constraints on the input. You will get some score for the correct submission of the subproblem. The description of the subproblems follows.

- In subproblem G1 (3 points), the constraints $1 \leq n \leq 6, 1 \leq k \leq 4$ will hold.
- In subproblem G2 (5 points), the constraints $1 \leq n \leq 30, 1 \leq k \leq 200$ will hold.
- In subproblem G3 (16 points), the constraints $1 \leq n \leq 100, 1 \leq k \leq 10^9$ will hold.

Output
Output the answer with absolute or relative error no more than $1e-9$.

input
3 1 1 2 3
output
0.8333333333333333

input
3 4 1 3 2
output
1.4583333333333334

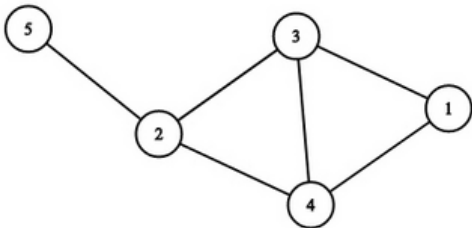
Consider the first sample test. We will randomly pick an interval of the permutation (1, 2, 3) (which has no inversions) and reverse the order of its elements. With probability $\frac{1}{2}$, the interval will consist of a single element and the permutation will not be altered. With probability $\frac{1}{6}$ we will inverse the first two elements' order and obtain the permutation (2, 1, 3) which has one inversion. With the same probability we might pick the interval consisting of the last two elements which will lead to the permutation (1, 3, 2) with one inversion. Finally, with probability $\frac{1}{6}$ the randomly picked interval will contain all elements, leading to the permutation (3, 2, 1) with 3 inversions. Hence, the expected number of inversions is equal to $\frac{1}{2} \cdot 0 + \frac{1}{6} \cdot 1 + \frac{1}{6} \cdot 1 + \frac{1}{6} \cdot 3 = \frac{5}{6}$.

G. The Maximum Subtree

2 seconds, 256 megabytes

Assume that you have k one-dimensional segments $s_1, s_2, \dots, s_k$ (each segment is denoted by two integers — its endpoints). Then you can build the following graph on these segments. The graph consists of k vertexes, and there is an edge between the i -th and the j -th vertexes ($i \neq j$) if and only if the segments s_i and s_j intersect (there exists at least one point that belongs to both of them).

For example, if $s_1 = [1, 6], s_2 = [8, 20], s_3 = [4, 10], s_4 = [2, 13], s_5 = [17, 18]$, then the resulting graph is the following:



A tree of size m is good if it is possible to choose m one-dimensional segments so that the graph built on these segments coincides with this tree.

You are given a tree, you have to find its good subtree with maximum possible size. Recall that a subtree is a connected subgraph of a tree.

Note that you have to answer q independent queries.

Input

The first line contains one integer q ($1 \leq q \leq 15 \cdot 10^4$) — the number of the queries.

The first line of each query contains one integer n ($2 \leq n \leq 3 \cdot 10^5$) — the number of vertices in the tree.

Each of the next $n - 1$ lines contains two integers x and y ($1 \leq x, y \leq n$) denoting an edge between vertices x and y . It is guaranteed that the given graph is a tree.

It is guaranteed that the sum of all n does not exceed $3 \cdot 10^5$.

Output

For each query print one integer — the maximum size of a good subtree of the given tree.

input
<pre>1 10 1 2 1 3 1 4 2 5 2 6 3 7 3 8 4 9 4 10</pre>
output
<pre>8</pre>

In the first query there is a good subtree of size 8. The vertices belonging to this subtree are 9, 4, 10, 2, 5, 1, 6, 3.

H. Constrained Tree

2 seconds, 256 megabytes

You need to find a binary tree of size n that satisfies a given set of c constraints. Suppose that the nodes of the unknown binary tree are labeled using a *pre-order* traversal starting with 1. For the i -th constraint you are given two labels, a_i and b_i and a direction, left or right. In case of left direction, b_i is an element of the subtree rooted at a_i 's left child. Similarly in the case of right direction b_i is an element of the subtree rooted at a_i 's right child.

Input

The first line of input contains two integers n and c . The next c lines contain 2 integers a_i, b_i ($1 \leq a_i, b_i \leq n$) and either "LEFT" or "RIGHT" denoting whether b is in the subtree rooted at a_i 's left child or in the subtree rooted at a_i 's right child.

The problem consists of multiple subproblems. The subproblems have different constraints on the input. You will get some score for the correct submission of the subproblem. The description of the subproblems follows.

- In subproblem D1 (9 points), the constraints $1 \leq n \leq 100$, $1 \leq c \leq 50$ will hold.
- In subproblem D2 (8 points), the constraints $1 \leq n \leq 1000000$, $1 \leq c \leq 100000$ will hold.

Output

Output will be on a single line.

Any binary tree that satisfies the constraints will be accepted. The tree's nodes should be printed out as n space separated labels representing an *in-order* traversal, using the *pre-order* numbers as labels of vertices.

If there are no trees that satisfy the constraints, print "IMPOSSIBLE" (without quotes).

input
<pre>3 2 1 2 LEFT 1 3 RIGHT</pre>
output
<pre>2 1 3</pre>

input
<pre>3 2 1 2 RIGHT 1 3 LEFT</pre>
output
<pre>IMPOSSIBLE</pre>

Consider the first sample test. We need to find a tree with 3 nodes that satisfies the following two constraints. The node labeled 2 with pre-order traversal should be in the left subtree of the node labeled 1 with pre-order traversal; the node labeled 3 with pre-order traversal should be in the right subtree of the node labeled 1. There is only one tree with three nodes that satisfies these constraints and its in-order traversal is (2, 1, 3).

Pre-order is the "root – left subtree – right subtree" order. *In-order* is the "left subtree – root – right subtree" order.

For other information regarding *in-order* and *pre-order*, see http://en.wikipedia.org/wiki/Tree_traversal.

I. Roads not only in Berland

2 seconds, 256 megabytes

Berland Government decided to improve relations with neighboring countries. First of all, it was decided to build new roads so that from each city of Berland and neighboring countries it became possible to reach all the others. There are n cities in Berland and neighboring countries in total and exactly $n - 1$ two-way roads. Because of the recent financial crisis, the Berland Government is strongly pressed for money, so to build a new road it has to close some of the existing ones. Every day it is possible to close one existing road and immediately build a new one. Your task is to determine how many days would be needed to rebuild roads so that from each city it became possible to reach all the others, and to draw a plan of closure of old roads and building of new ones.

Input

The first line contains integer n ($2 \leq n \leq 1000$) — amount of cities in Berland and neighboring countries. Next $n - 1$ lines contain the description of roads. Each road is described by two space-separated integers a_i, b_i ($1 \leq a_i, b_i \leq n, a_i \neq b_i$) — pair of cities, which the road connects. It can't be more than one road between a pair of cities. No road connects the city with itself.

Output

Output the answer, number t — what is the least amount of days needed to rebuild roads so that from each city it became possible to reach all the others. Then output t lines — the plan of closure of old roads and building of new ones. Each line should describe one day in the format $i \ j \ u \ v$ — it means that road between cities i and j became closed and a new road between cities u and v is built. Cities are numbered from 1. If the answer is not unique, output any.

input
2 1 2
output
0

input
7 1 2 2 3 3 1 4 5 5 6 6 7
output
1 3 1 3 7

J. Valuable Paper

2 seconds, 256 megabytes

The pandemic is upon us, and the world is in shortage of the most important resource: toilet paper. As one of the best prepared nations for this crisis, BubbleLand promised to help all other world nations with this valuable resource. To do that, the country will send airplanes to other countries carrying toilet paper.

In BubbleLand, there are N toilet paper factories, and N airports. Because of how much it takes to build a road, and of course legal issues, every factory must send paper to only one airport, and every airport can only take toilet paper from one factory.

Also, a road can't be built between all airport-factory pairs, again because of legal issues. Every possible road has number d given, number of days it takes to build that road.

Your job is to choose N factory-airport pairs, such that if the country starts building all roads at the same time, it takes the least amount of days to complete them.

Input

The first line contains two integers N ($1 \leq N \leq 10^4$) - number of airports/factories, and M ($1 \leq M \leq 10^5$) - number of available pairs to build a road between.

On next M lines, there are three integers u, v ($1 \leq u, v \leq N$), d ($1 \leq d \leq 10^9$) - meaning that you can build a road between airport u and factory v for d days.

Output

If there are no solutions, output -1. If there exists a solution, output the minimal number of days to complete all roads, equal to maximal d among all chosen roads.

input
<pre>3 5 1 2 1 2 3 2 3 3 3 2 1 4 2 2 5</pre>
output
4

K. Bill Total Value

1 second, 256 megabytes

Vasily exited from a store and now he wants to recheck the total price of all purchases in his bill. The bill is a string in which the names of the purchases and their prices are printed in a row without any spaces. Check has the format " $name_1price_1name_2price_2...name_nprice_n$ ", where $name_i$ (name of the i -th purchase) is a non-empty string of length not more than 10, consisting of lowercase English letters, and $price_i$ (the price of the i -th purchase) is a non-empty string, consisting of digits and dots (decimal points). It is possible that purchases with equal names have different prices.

The price of each purchase is written in the following format. If the price is an integer number of dollars then cents are not written.

Otherwise, after the number of dollars a dot (decimal point) is written followed by cents **in a two-digit format** (if number of cents is between 1 and 9 inclusively, there is a leading zero).

Also, every three digits (from less significant to the most) in dollars are separated by dot (decimal point). No extra leading zeroes are allowed. The price always starts with a digit and ends with a digit.

For example:

- "234", "1.544", "149.431.10", "0.99" and "123.05" are valid prices,
- ",.333", "3.33.11", "12.00", ".33", "0.1234" and "1.2" are not valid.

Write a program that will find the total price of all purchases in the given bill.

Input

The only line of the input contains a non-empty string s with length not greater than 1000 — the content of the bill.

It is guaranteed that the bill meets the format described above. It is guaranteed that each price in the bill is not less than one cent and not greater than 10^6 dollars.

Output

Print the total price **exactly in the same format** as prices given in the input.

input
chipsy48.32televizor12.390
output
12.438.32

input
a1b2c3.38
output
6.38

input
aa0.01t0.03
output
0.04

L. Second price auction

2 seconds, 256 megabytes

Nowadays, most of the internet advertisements are not statically linked to a web page. Instead, what will be shown to the person opening a web page is determined within 100 milliseconds after the web page is opened. Usually, multiple companies compete for each ad slot on the web page in an auction. Each of them receives a request with details about the user, web page and ad slot and they have to respond within those 100 milliseconds with a bid they would pay for putting an advertisement on that ad slot. The company that suggests the highest bid wins the auction and gets to place its advertisement. If there are several companies tied for the highest bid, the winner gets picked at random.

However, the company that won the auction does not have to pay the exact amount of its bid. In most of the cases, a second-price auction is used. This means that the amount paid by the company is equal to the maximum of all the other bids placed for this ad slot.

Let's consider one such bidding. There are n companies competing for placing an ad. The i -th of these companies will bid an integer number of microdollars equiprobably randomly chosen from the range between L_i and R_i , inclusive. In the other words, the value of the i -th company bid can be any integer from the range $[L_i, R_i]$ with the same probability.

Determine the expected value that the winner will have to pay in a second-price auction.

Input

The first line of input contains an integer number n ($2 \leq n \leq 5$). n lines follow, the i -th of them containing two numbers L_i and R_i ($1 \leq L_i \leq R_i \leq 10000$) describing the i -th company's bid preferences.

This problem doesn't have subproblems. You will get 8 points for the correct submission.

Output

Output the answer with absolute or relative error no more than $1e-9$.

input
3 4 7 8 10 5 5
output
5.7500000000

input
3 2 5 3 4 1 6
output
3.5000000000

Consider the first example. The first company bids a random integer number of microdollars in range $[4, 7]$; the second company bids between 8 and 10, and the third company bids 5 microdollars. The second company will win regardless of the exact value it bids, however the price it will pay depends on the value of first company's bid. With probability 0.5 the first company will bid at most 5 microdollars, and the second-highest price of the whole auction will be 5. With probability 0.25 it will bid 6 microdollars, and with probability 0.25 it will bid 7 microdollars. Thus, the expected value the second company will have to pay is $0.5 \cdot 5 + 0.25 \cdot 6 + 0.25 \cdot 7 = 5.75$.

M. Edges in MST

2 seconds, 256 megabytes

You are given a connected weighted undirected graph without any loops and multiple edges.

Let us remind you that a graph's spanning tree is defined as an acyclic connected subgraph of the given graph that includes all of the graph's vertexes. The weight of a tree is defined as the sum of weights of the edges that the given tree contains. The minimum spanning tree (MST) of a graph is defined as the graph's spanning tree having the minimum possible weight. For any connected graph obviously exists the minimum spanning tree, but in the general case, a graph's minimum spanning tree is not unique.

Your task is to determine the following for each edge of the given graph: whether it is either included in **any** MST, or included **at least in one** MST, or **not included in any** MST.

Input

The first line contains two integers n and m ($2 \leq n \leq 10^5$, $n - 1 \leq m \leq \min(10^5, \frac{n(n-1)}{2})$) — the number of the graph's vertexes and edges, correspondingly. Then follow m lines, each of them contains three integers — the description of the graph's edges as " $a_i b_i w_i$ " ($1 \leq a_i, b_i \leq n$, $1 \leq w_i \leq 10^6$, $a_i \neq b_i$), where a_i and b_i are the numbers of vertexes connected by the i -th edge, w_i is the edge's weight. It is guaranteed that the graph is connected and doesn't contain loops or multiple edges.

Output

Print m lines — the answers for all edges. If the i -th edge is included in any MST, print "any"; if the i -th edge is included at least in one MST, print "at least one"; if the i -th edge isn't included in any MST, print "none". Print the answers for the edges in the order in which the edges are specified in the input.

input
4 5 1 2 101 1 3 100 2 3 2 2 4 2 3 4 1

output

none
any
at least one
at least one
any

input

3 3
1 2 1
2 3 1
1 3 2

output

any
any
none

input

3 3
1 2 1
2 3 1
1 3 1

output

at least one
at least one
at least one

In the second sample the MST is unique for the given graph: it contains two first edges.

In the third sample any two edges form the MST for the given graph. That means that each edge is included at least in one MST.